Phase 1 Repository Discovery

Corrected Implementation Summary and Phase 2 Handoff

Project: AI-Powered GitHub Repository Modernization Platform

Prepared: 24 Jun 2026

This document records only the corrected Phase 1 state after GitHub token setup. It excludes earlier incorrect dependency commands and keeps the final workspace-safe implementation that should be used as context for Phase 2.

Item	Final Phase 1 Position
Phase	Phase 1 - Repository Discovery
Primary goal	Connect to GitHub, enumerate repositories, persist metadata locally, and expose discovery output to later phases.
Architecture rule	Octokit is isolated in packages/github-client. Other packages consume @repo/github-client only.
Storage rule	Repository and Run records are persisted through @repo/db using Prisma and PostgreSQL.
Execution rule	Discovery is run through the @repo/tools CLI wrapper.
Phase 2 dependency	Phase 2 should reuse Repository rows plus getRepoTree/getFileContent from @repo/github-client.

1. Phase 1 Scope

Phase 1 is limited to repository discovery. It does not implement the deterministic auditor, stack detection, LLM workflows, security agent, dependency intelligence, dashboard, or PR automation.

Implemented scope

- Load GitHub credentials from local environment variables.
- Create an isolated GitHub client package that wraps Octokit REST and GraphQL.
- Fetch authenticated user repositories and optional organization repositories.
- Support filters for forks, archived repositories, minimum stars, and topics.
- Persist discovered repository metadata into PostgreSQL through Prisma.
- Create a Run record for every discovery execution and mark it success or failed.
- Compute basic derived metrics used by later phases: daysSinceLastPush, hasTopics, isStale, and starVelocity where possible.
- Expose a CLI command for repeatable local execution.
- Fix pnpm workspace dependency resolution so local packages are not fetched from npm registry.

Explicit non-scope

- No repository audit findings yet.
- No README/LICENSE/.gitignore checks yet.
- No secret scanning beyond future Phase 2/8 planning.
- No GitHub write permissions, commits, pull requests, or merge automation.
- No GitHub App auth implementation yet; local development uses a fine-grained personal access token.
- No dashboard or API server changes are required for Phase 1 completion.

2. Corrected Token Handling

The original token visible in a screenshot must be considered compromised and revoked. The correct final state is to generate a new fine-grained personal access token and store it only in a local .env file that is ignored by Git.

```
# .env - local only, never commit
GITHUB_TOKEN="your_new_token_here"
GITHUB_OWNER=""
```

Minimum fine-grained token permissions for Phase 1

- Repository access: all repositories, or selected repositories for testing.
- Repository permissions: Metadata read-only.
- Repository permissions: Contents read-only, required for tree and file content reads used by Phase 1 and Phase 2.
- No write permissions are required in Phase 1.

3. Workspace and Dependency Corrections

The main issue encountered during Phase 1 was pnpm attempting to fetch local workspace packages such as @repo/db from the public npm registry. The corrected rule is to use the workspace protocol for all internal packages.

Problem	Cause	Correct Fix
ERR_PNPM_FETCH_404 for @repo/db	@repo/db was being treated as an npm registry package.	Reference it as @repo/db: workspace:* inside packages/repo-scanner/package.json.
tools package not detected	pnpm-workspace.yaml used tools/*, but the package is tools/package.json.	Use tools in pnpm-workspace.yaml, not only tools/*.
@repo/db main output mismatch	package.json pointed main to ./dist/index.ts.	Use ./dist/index.js because build output is JavaScript.
Runtime libs installed at root	Package-specific dependencies were not scoped to the package that uses them.	Install Octokit only in @repo/github-client. Install dev tooling at root only when shared.

Correct pnpm-workspace.yaml

```
packages:
  - "apps/*"
  - "packages/*"
  - "tools"
  - "examples/*"
```

4. Final Package Structure

```
ai_powered_github_repository_modernization_platform/
  packages/
    core-types/
      package.json
      tsconfig.json
      src/index.ts
    db/
      package.json
      tsconfig.json
      prisma.config.ts
      prisma/schema.prisma
      src/index.ts
    github-client/
      package.json
      tsconfig.json
      src/index.ts
      src/types.ts
      src/errors.ts
      src/rate-limit.ts
      src/rate-limit.test.ts
      src/github-client.ts
    repo-scanner/
      package.json
      tsconfig.json
      src/index.ts
      src/metrics.ts
      src/metrics.test.ts
      src/discovery.ts
  tools/
    package.json
    tsconfig.json
    discover-repos.ts
  .env.example
  .gitignore
  package.json
  pnpm-workspace.yaml
  pnpm-lock.yaml
```

5. Correct Final package.json Files

5.1 packages/db/package.json

```
{
  "name": "@repo/db",
  "private": true,
  "version": "0.1.0",
  "type": "module",
  "main": "./dist/index.js",
  "types": "./dist/index.d.ts",
  "scripts": {
    "build": "tsc -p tsconfig.json",
    "test": "vitest run --passWithNoTests",
    "lint": "eslint .",
    "typecheck": "tsc -p tsconfig.json --noEmit",
    "db:generate": "prisma generate --schema ./prisma/schema.prisma",
    "db:migrate": "prisma migrate dev --schema ./prisma/schema.prisma"
  },
  "dependencies": {
    "@prisma/adapter-pg": "^7.8.0",
    "@prisma/client": "^7.8.0",
    "pg": "^8.22.0"
  },
  "devDependencies": {
    "@types/pg": "^8.20.0",
    "dotenv": "^17.4.2",
    "prisma": "^7.8.0"
  }
}
```

5.2 packages/github-client/package.json

```
{
  "name": "@repo/github-client",
  "version": "0.1.0",
  "private": true,
  "type": "module",
  "main": "./dist/index.js",
  "types": "./dist/index.d.ts",
  "exports": {
    ".": {
      "types": "./dist/index.d.ts",
      "default": "./dist/index.js"
    }
  },
  "scripts": {
    "build": "tsc -p tsconfig.json",
    "typecheck": "tsc -p tsconfig.json --noEmit",
    "test": "vitest run",
    "lint": "eslint ."
  },
  "dependencies": {
    "@octokit/graphql": "latest",
    "@octokit/rest": "latest"
  },
  "devDependencies": {
    "vitest": "latest"
  }
}
```

5.3 packages/repo-scanner/package.json

```
{
  "name": "@repo/repo-scanner",
  "version": "0.1.0",
  "private": true,
  "main": "./dist/index.js",
  "types": "./dist/index.d.ts",
  "type": "module",
  "scripts": {
    "build": "tsc -p tsconfig.json",
    "test": "vitest run --passWithNoTests",
    "lint": "eslint .",
    "typecheck": "tsc -p tsconfig.json --noEmit"
  },
  "dependencies": {
    "@repo/core-types": "workspace:*",
    "@repo/db": "workspace:*",
    "@repo/github-client": "workspace:*"
  }
}
```

5.4 tools/package.json

```
{
  "name": "@repo/tools",
  "version": "0.1.0",
  "private": true,
  "type": "module",
  "scripts": {
    "discover:repos": "tsx discover-repos.ts",
    "build": "tsc -p tsconfig.json",
    "typecheck": "tsc -p tsconfig.json --noEmit"
  },
  "dependencies": {
    "@repo/repo-scanner": "workspace:*",
    "dotenv": "^17.4.2"
  },
  "devDependencies": {
    "tsx": "latest"
  }
}
```

5.5 Root package.json script addition

```
{
  "scripts": {
    "discover:repos": "pnpm --filter @repo/tools discover:repos"
  }
}
```

6. Database Schema Required by Phase 1

The Repository model stores GitHub metadata needed by Phase 1 and consumed by Phase 2. The Run model records discovery executions for traceability and later analytics.

```
model Repository {
  id             String    @id @default(uuid())
  nameWithOwner  String    @unique
  name           String
  owner          String
  isFork         Boolean   @default(false)
  isArchived     Boolean   @default(false)
  stars          Int       @default(0)
  forks          Int       @default(0)
  primaryLanguage String?
  topics         String[]
  pushedAt       DateTime
  githubUpdatedAt DateTime?
  lastScannedAt  DateTime?
  createdAt      DateTime  @default(now())
  updatedAtLocal  DateTime  @updatedAt
}

model Run {
  id         String    @id @default(uuid())
  type       String
  startedAt  DateTime  @default(now())
  finishedAt DateTime?
  status     String
  repoCount  Int       @default(0)
  error      String?
}
```

7. GitHub Client Contract

The GitHub client package is the only layer allowed to import Octokit. This prevents API access from spreading into scanner, auditor, dashboard, or future agent packages.

Public interface

```
export interface GithubClient {
  listUserRepos(opts?: ListReposOptions): Promise<RepoSummary[]>;
  listOrgRepos(org: string, opts?: ListReposOptions): Promise<RepoSummary[]>;
  getRepoTree(owner: string, repo: string, ref?: string): Promise<TreeEntry[]>;
  getFileContent(owner: string, repo: string, path: string): Promise<string | null>;
  getRateLimitStatus(): Promise<RateLimitStatus>;
}
```

Key implementation decisions

- GraphQL is used for paginated repository listing so stars, forks, topics, language, pushed date, and archived/fork status can be fetched efficiently.
- REST is used for getRepoTree and getFileContent because these operations map cleanly to GitHub repository content/tree endpoints.
- Rate-limit handling checks GitHub remaining quota and waits when the remaining count is near zero.
- Errors are wrapped with package-specific GithubClientError classes where helpful.

8. Repository Scanner Contract

The repository scanner is the orchestration layer for discovery. It does not talk directly to Octokit; it consumes @repo/github-client and @repo/db.

Discovery flow

Step	Action	Output
1	Create Run row	Run status = running
2	Fetch repos from user or org	RepoSummary[]
3	Find existing repository by nameWithOwner	Previous snapshot for star velocity
4	Compute derived metrics	daysSinceLastPush, hasTopics, isStale, starVelocity
5	Upsert Repository	No duplicate rows on repeated discovery
6	Update Run row	status = success or failed, repoCount, error if any

Derived metrics

Metric	Formula / Meaning	Phase 2+ Use
daysSinceLastPush	now - pushedAt, in days	Health score freshness deduction
hasTopics	topics.length > 0	Portfolio and metadata quality
isStale	daysSinceLastPush > 180	Freshness flag for HealthScore
starVelocity	stars delta / days between runs when previous scan exists	Portfolio optimizer later

9. CLI Execution Contract

The CLI should be run from the root through the root script. The tools package owns the actual discover-repos.ts file.

```
# Basic discovery
pnpm discover:repos

# Include forked repositories
pnpm discover:repos --include-forks

# Include archived repositories
pnpm discover:repos --include-archived

# Minimum star filter
pnpm discover:repos --min-stars=5

# Topic filter
pnpm discover:repos --topics=ai,typescript,automation

# Organization discovery
pnpm discover:repos --org=your-org-name
```

10. Installation and Validation Commands

Use these commands after the package.json and workspace corrections are in place.

```
# Install workspace dependencies from root
pnpm install

# Generate and migrate Prisma client/schema
pnpm --filter @repo/db db:generate
pnpm --filter @repo/db db:migrate

# Run discovery
pnpm discover:repos

# Inspect data manually
pnpm --filter @repo/db prisma studio

# Full validation
pnpm format:check
pnpm lint
pnpm typecheck
pnpm build
pnpm test
```

Expected database result

- Repository table contains one row per discovered repository.
- nameWithOwner is unique and prevents duplicates.
- Run table receives one row per CLI execution.
- Running discovery twice should not duplicate Repository rows, but should create two Run rows.
- Run.status must end as success or failed, never remain running after completion/error handling.

11. Phase 1 Done Checklist

[ ] GITHUB_TOKEN is loaded from local .env and is not committed.

[ ] @repo/github-client connects to GitHub successfully.

[ ] Octokit is used only inside packages/github-client.

[ ] GraphQL repository pagination works for user repository discovery.

[ ] Optional organization repository discovery works when token permissions allow it.

[ ] Repository metadata persists into Prisma/PostgreSQL.

[ ] Repository upsert prevents duplicates on repeated discovery runs.

- [ ] Run table records every discovery execution.
- [ ] Derived metrics are computed during persistence.
- [ ] Rate-limit helper exists and is tested.
- [ ] pnpm discover:repos works from the repository root.
- [ ] pnpm lint passes.
- [ ] pnpm typecheck passes.
- [ ] pnpm build passes.
- [ ] pnpm test passes.
- [ ] Phase 1 commit is created: feat: implement phase 1 repository discovery.

12. Known Limitations Carried Forward

- GitHub App authentication is not implemented yet. Phase 1 local development uses a fine-grained personal access token.
- Token encryption/secrets table is not implemented yet. Local mode uses environment variables only.
- starVelocity is meaningful only after at least two discovery runs with different lastScannedAt values.
- Rate-limit behavior has a helper/test, but should be mocked more thoroughly if quota-sensitive behavior becomes important.
- Repository content has not been audited yet; Phase 1 only prepares getRepoTree and getFileContent for later use.
- No findings or health scores are created in Phase 1.

13. Phase 2 Handoff Summary

Phase 2 - Deterministic Repository Auditor should start from the Phase 1 repository inventory. It should not re-implement repository discovery or import Octokit directly.

Phase 2 can rely on these Phase 1 outputs

Available From Phase 1	How Phase 2 Should Use It
Repository rows	Iterate over repositories to audit owner/name/nameWithOwner, freshness, topics, language, stars, and forks.
Run rows	Create new Run rows with type = audit for Phase 2 executions.
getRepoTree(owner, repo, ref?)	Fetch tree entries to detect README, LICENSE, .gitignore, node_modules, .env files, and large files.
getFileContent(owner, repo, path)	Read README length, .gitignore contents, LICENSE contents, and lightweight .env evidence where needed.
daysSinceLastPush / isStale	Use freshness as part of the HealthScore calculation.
Workspace package rules	Create @repo/repo-auditor or similar and depend on internal packages with workspace:* only.

Recommended Phase 2 package

```
packages/repo-auditor/
  package.json
  tsconfig.json
  src/index.ts
  src/audit.ts
  src/rules/readme.ts
  src/rules/license.ts
  src/rules/gitignore.ts
  src/rules/tracked-directories.ts
  src/rules/env-exposure.ts
  src/rules/large-files.ts
  src/health-score.ts
  src/findings.ts
  src/fixtures/
```

Phase 2 models to add/complete

```
model Finding {
  id           String   @id @default(uuid())
  repositoryId String
  repository   Repository @relation(fields: [repositoryId], references: [id])
  category     String
  ruleId       String
  severity     String
  confidence   Float    @default(1.0)
  message      String
  filePath     String?
  detectedAt   DateTime @default(now())
  resolvedAt   DateTime?

  @@unique([repositoryId, ruleId, filePath])
}

model HealthScore {
  id           String   @id @default(uuid())
  repositoryId String
  repository   Repository @relation(fields: [repositoryId], references: [id])
  score        Int
  breakdown    Json
  computedAt   DateTime @default(now())
}
```

14. Phase 2 Implementation Targets

Phase 2 Target	Required Behavior
README detection	Check README* case-insensitively at root. Fetch content and flag stub README if under 200 characters.
LICENSE detection	Check LICENSE* case-insensitively. Attempt SPDX-based classification without LLM usage.
.gitignore detection	Check root .gitignore and read contents for later cross-checks.
Tracked dependency folders	Detect committed node_modules, vendor, .venv, and __pycache__ paths in the Git tree.
.env exposure	Detect .env and .env.* while excluding .env.example, .env.sample, and .env.template.
Large files	Flag files over 5MB, 25MB, and 95MB with increasing severity.
HealthScore	Compute deterministic score from hygiene, security exposure, structure, and freshness. Dependency health remains placeholder until Phase 9.
Idempotency	Do not duplicate open findings on repeated audits. Upsert by repositoryId + ruleId + filePath.

Suggested Phase 2 root script

```
{
  "scripts": {
    "audit:repos": "pnpm --filter @repo/tools audit:repos"
  }
}
```

Suggested Phase 2 package dependencies

```
{
  "dependencies": {
    "@repo/core-types": "workspace:*",
    "@repo/db": "workspace:*",
    "@repo/github-client": "workspace:*",
    "spdx-license-list": "latest"
  }
}
```

15. Final Rule for Next Development

From this point forward, every new internal package should use workspace:* for internal dependencies. Every external runtime dependency should be installed only in the package that directly imports it. Shared dev tooling can remain at the root workspace.

Recommended Phase 1 commit message

```
git add .
git commit -m "feat: implement phase 1 repository discovery"
```